Декартовы деревья и range-деревья

Содержание

1 Декартово дерево **2**
1.1 Мотивация и основная идея 2
1.2 Операции split и merge 2
1.3 Построение за $O(n)$ 3
1.4 Случайные приоритеты и высота 4
1.5 Декартово дерево по неявному ключу 4

2 Range-деревья **5**
2.1 Постановка задачи 5
2.2 Одномерный случай 5
2.3 Двумерный случай 6
2.4 Алгоритм запроса 7
2.5 Многомерные range-деревья 7
2.6 Fractional Cascading и Layered Range-Tree 8
2.7 Совпадающие координаты 10

1 Декартово дерево

1.1 Мотивация и основная идея

Обычное BST (двоичное дерево поиска) без балансировки может выродиться в список, из-за чего операции поиска/вставки/удаления и т.д. будут работать долго. Хотим придумать структуру с гарантированной средней глубиной $O(\log n)$.

Определение. *Декартово дерево (ДД)* — бинарное дерево, в узлах которого хранятся пары (x, y), являющееся одновременно BST по x и кучей по y. Координата x называется *ключом*, а координата y — приоритетом. Для удобства в дальнейшем давайте считать, что куча по y - это max-куча.

Утверждение. *При различных ключах и приоритетах декартово дерево* ***единственно****.*

Доказательство. Корнем точно будет вершина с максимальным y, оставшиеся вершины разбиваем на группы с ключом меньше ключа корня и больше ключа корня, для них рекурсивно повторяем рассуждение. □

1.2 Операции split и merge

Все операции сводятся к двум базовым операциям (**split** и **merge**). Каждая из этих операций будет делать единственный рекурсивный вызов от дерева глубины на ≥ 1 меньше, а значит они будут работать за $O(h)$, где h — глубина.

split$(T, k) \to \langle L, R\rangle$ — разрезать ДД T по ключу k, то есть разбить на два ДД: в L должны попасть вершины с ключами $< k$, в R — остальные.

Работает **split** так: если $k > x_{\text{root}}$ — рекурсия уходит в правое поддерево (см. рис. 1), иначе — в левое.

merge$(L, R) \to T$ — объединить два ДД при условии, что ключи $L <$ ключи R.

Устройство **merge**: пусть приоритет корня L больше приоритета корня R (другой случай аналогичен), его возьмем корнем. Тогда в качестве итогового ДД подойдет такое дерево: к уже выбранному корню слева прикрепляем левое поддерево L, а справа прикрепляем результат **merge** R и правого поддерева L (см. рис. 2).

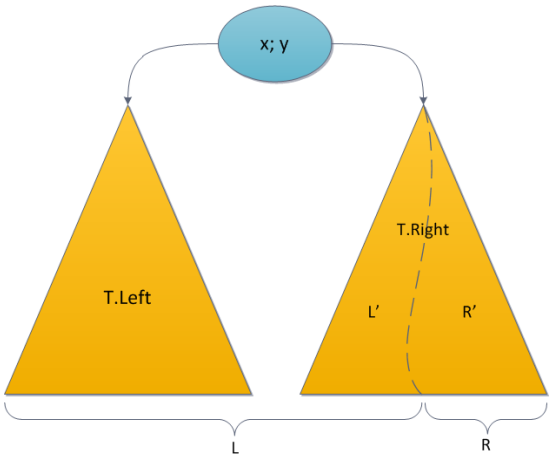

Рис. 1: Операция **split**

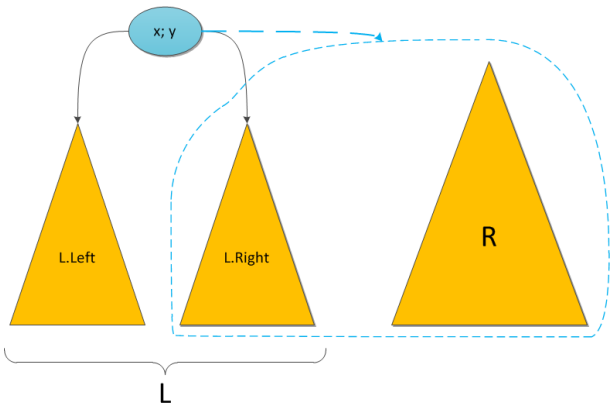

Рис. 2: Операция **merge**

Из полученных операций можно легко реализовать интерфейс **set**:

insert(T, v) — добавить вершину v (ключ и приоритет):

Делаем split по $v.x$ (получим L и R), после делаем слияние L и v (из одной вершины всегда можно построить дерево), результат нужно слить с R.

remove(T, x) — удалить вершину с ключом x:

Делаем split по x (получим L и R), удаляем у R самую левую вершину, если её ключ равен x (получим R'), сливаем L и R'.

1.3 Построение за $O(n)$

Данный алгоритм работает только в случае, когда точки $(x_1, y_1), ..., (x_n, y_n)$ уже отсортированы по x (по возрастанию). В противном случае время работы $O(n \log n)$ из-за сортировки.

Строим итеративно слева направо, при вставке очередного (x_k, y_k) идем по самой правой ветви дерева снизу вверх и находим первую вершину v с приоритетом больше y_k. Если такой нет, текущее дерево делаем левым поддеревом (x_k, y_k); иначе на этой итерации изменим дерево так: вершине (x_k, y_k) слева подвесим правое поддерево v, а справа - ничего; вершине v вместо правого поддерева подвесим вершину (x_k, y_k).

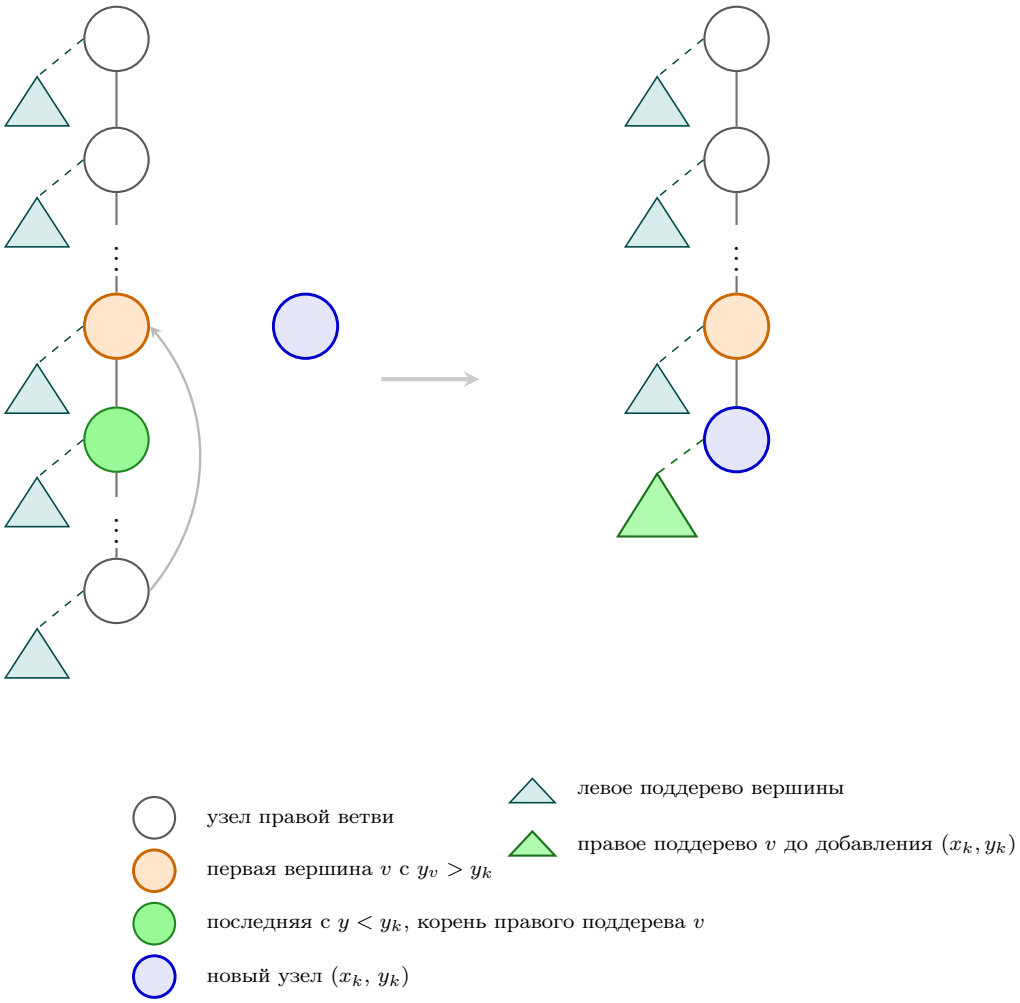

Рис. 3: Шаг алгоритма построения ДД за $O(n)$: вставка узла (x_k, y_k). Серая стрелка показывает подъём по правой ветви снизу вверх до первой вершины v с $y_v > y_k$.

При таком алгоритме все вершины посещены ≤ 2 раз (при добавлении 1 раз, при втором посещении мы ее отправляем в левое поддерево, а алгоритм всегда шагает по самой правой ветке), время работы: $O(n)$.

1.4 Случайные приоритеты и высота

Теорема. *В декартовом дереве из n узлов со случайными одинаково распределенными приоритетами верно: $\mathbb{E}[d(x_k)] = O(\log n)$, где $d(x_k)$ - глубина k-й вершины.*

Доказательство. Для удобства $x_1 < x_2 < ... < x_n$.

1. Индикатор $A_{i,k} := 1 \Leftrightarrow x_i$ — предок x_k. Тогда $d(x_k) = \sum_i A_{i,k}$, и по линейности матожидания $\mathbb{E}[d(x_k)] = \sum_i \Pr[A_{i,k} = 1]$.

2. **Лемма.** x_i — предок $x_k \Leftrightarrow x_i$ имеет наибольший приоритет в $X_{i,k} = \{x_{\min(i,k)}, \ldots, x_{\max(i,k)}\}$.

   Доказательство (индукция по размеру поддерева): если корень — x_i, то он предок и имеет максимальный приоритет; если корень — x_k, то x_i не предок и не максимален в $X_{i,k}$; если корень — x_m между i и k, то $x_m \in X_{i,k}$, и x_i не предок; иначе — рекурсия на меньшее поддерево.

3. Из равномерности: $\mathbb{P}[A_{i,k} = 1] = \dfrac{1}{|k-i|+1}$.

4. Итого: $\mathbb{E}[d(x_k)] = \displaystyle\sum_{i \neq k} \frac{1}{|k-i|+1} \leq \ln k + 1 + \ln(n-k) + 1 = O(\log n)$. □

Следствие. *Среднее время указанных операций над ДД — $O(\log n)$.*

1.5 Декартово дерево по неявному ключу

Мотивация: Хотим эффективно реализовать динамический массив с вставкой/удалением в произвольную позицию и операциями на отрезке.

Идея: Не будем хранить ключ x явно, но будем подразумевать, что в вершинах ДД он есть автоматически. Для удобства можно считать, что в дереве из n вершин ключами будут числа $0, 1, ..., n-1$. Заметим, что их расположение в вершинах дерева однозначно восстанавливается по структуре дерева. Значит можно считать наше дерево массивом в том смысле, что на месте i будет стоять элемент, у которого есть неявный ключ i.

Операции:
merge(L, R) - дописать в конец массива L массив R. Заметим, что реализация **merge** для обычного ДД не пользовалась ключами, а лишь предполагала, что ключи в $L <$ ключей R. Тогда эта реализация подойдет.

Для реализации операции **split** нужно хранить дополнительную информацию: в каждой вершине хранится C — размер поддерева.
split(T, k) теперь отрезает ровно k вершин от ДД T: если $l = |T.left| \geq k$ — рекурсия в левом поддереве с параметром k; иначе — в правом с параметром $k - l - 1$. После изменений детей как в обычном **split** обновляем размеры поддеревьев там, где они поменялись. Сложность времени работы не изменилась.

Применения: логарифмическое время на все следующее: вставка на фиксированную позицию, удаление, перестановка подотрезка, массовые операции на отрезке (аналогично ДО).

2 Range-деревья

2.1 Постановка задачи

Дано множество из n точек в $\mathbb{R}^d$; хотим отвечать на запросы вида «найти все точки в прямоугольном гиперпараллелепипеде $[x_1 : x_1'] \times [x_2 : x_2'] \times \ldots \times [x_d : x_d']$». Рассмотрим **range-деревья**, которые имеют ряд преимуществ в сравнении с другими известными структурами данных (например kd-tree). Итоговая сложность будет такая:

Структура	Построение	Память	Запрос
kd-tree	$O(n \log n)$	$O(n)$	$O(n^{1-\frac{1}{d}} + k)$
Range-tree	$O(n \log^{d-1} n)$	$O(n \log^{d-1} n)$	$O(\log^d n + k)$
Layered range-tree	$O(n \log^{d-1} n)$	$O(n \log^{d-1} n)$	$O(\log^{d-1} n + k)$

Здесь k — число найденных точек в гиперпараллелепипеде для данного запроса.

2.2 Одномерный случай

Прежде чем перейти к многомерным range-деревьям, разберём одномерный случай: дано множество $P = \{p_1, \ldots, p_n\}$ вещественных чисел, запросы вида $[x : x']$.

Структура данных. Храним P в листьях сбалансированного BST T. Внутренние вершины хранят значение для разделения, листья — сами точки.

Определение. *Каноническое подмножество* вершины дерева — множество листьев её поддерева. Для вершины ν ее каноническое подмножество будем обозначать $P(\nu)$.

Алгоритм запроса 1DRangeQuery$(T, [x : x'])$:

1. Делаем два поиска в BST для x и x', находим ν_{split} — вершину, где пути поиска x и x' расходятся.
2. Идём по пути к x: на каждом шаге влево возвращаем листья правого поддерева.
3. Идём по пути к x': на каждом шаге вправо возвращаем листья левого поддерева.
4. Проверяем листья путей поиска, в которых пути заканчиваются (μ и μ').

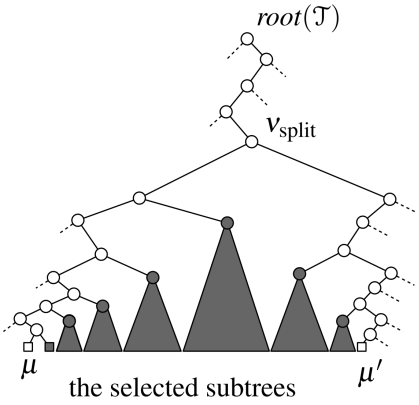

Рис. 4: 1DRangeQuery

Теорема. *Множество P из n точек в $\mathbb{R}^1$ можно хранить в сбалансированном BST, использующем $O(n)$ памяти и строящемся за $O(n \log n)$ так, что ответ на запрос $[x : x']$ тратит $O(\log n + k)$ времени, где k — число найденных точек запроса.*

Доказательство. Корректность очевидна, время работы следует из того, что поиск в BST тратит $O(\log n)$, а возврат k найденных точек из листьев займет $O(k)$, так как в поддереве листья занимают $\geq \frac{1}{2}$ всех вершин поддерева. □

Замечание. **Важно отметить**, что возвращений левых/правых поддеревьев будет $O(\log n)$ (в силу сбалансированности дерева), то есть любой отрезок представляется в виде дизъюнктного объединения канонических подмножеств $O(\log n)$ вершин, при том эти вершины мы находим за $O(\log n)$.

2.3 Двумерный случай

Главная идея: превратить двумерный запрос $[x : x'] \times [y : y']$ в два одномерных (с которыми мы уже умеем работать!).

Определение. *Range-дерево* для набора P из n точек плоскости — двухуровневая структура:

- *Главное дерево* T — сбалансированный BST по x-координатам;
- Каждая вершина $\nu \in T$ хранит указатель на *ассоциированную структуру* $T_{\text{assoc}}(\nu)$ — сбалансированное BST по y-координатам точек из канонического подмножества — $P(\nu)$.

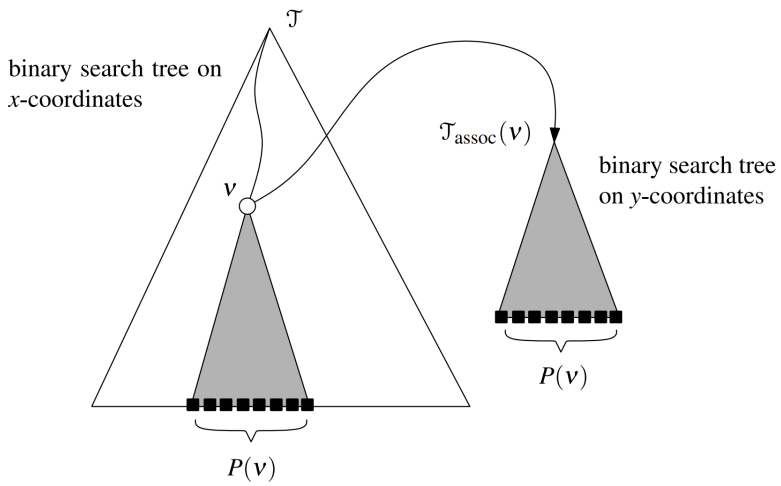

Рис. 5: 2D Range-дерево

Алгоритм построения Build2DRangeTree(P): рекурсивно строим главное дерево по x. В каждой вершине сначала строим T_{assoc} по y, после рекурсивно строим для точек с x меньше медианы и больше медианы, после к точке с медианным x слева и справа подвешиваем результаты работы рекурсии, а ассоциированной структурой для нее назначим построенный ранее T_{assoc}. При всем этом поддерживаем два отсортированных списка — по x и по y для $P(\nu)$ для всех ν. Тогда T_{assoc} для P строится за $O(|P|)$ (как в слиянии в mergesort).

Лемма. *Range-дерево для n точек плоскости использует $O(n \log n)$ памяти и строится за $O(n \log n)$.*

Доказательство. Точка p хранится ровно в одном $T_{\text{assoc}}(\nu)$ для каждого уровня глубины. Уровней $O(\log n)$, на каждом суммарно $O(n)$ точек, значит суммарная память $O(n \log n)$. Каждый лист на любой глубине участвует в $O(1)$ операциях, значит аналогично время работы $O(n \log n)$ (еще сортировка по x в самом начале, но она не влияет на временную сложность). □

2.4 Алгоритм запроса

Сначала выбираем точки с x-координатой в $[x : x']$ как в 1DRangeQuery (но пока ничего не возвращаем) — это $O(\log n)$ канонических подмножеств из BST по x. Затем в каждом из них ищем точки с y-координатой в $[y : y']$ с помощью вспомогательных BST по y, здесь уже просто стандартный 1DRangeQuery.

Лемма. *Запрос к двумерному range-дереву из n точек работает за $O(\log^2 n + k)$.*

Доказательство. Главное дерево посещает $O(\log n)$ вершин на каждом из двух путей поиска. В каждой из $O(\log n)$ выбранных вершин ν выполняется 1DRangeQuery за $O(\log n + k_\nu)$. Суммарно: $\sum_\nu O(\log n + k_\nu) = O(\log^2 n) + O(k)$. □

Теорема. *Range-дерево для n точек плоскости использует $O(n \log n)$ памяти, строится за $O(n \log n)$, ответ на запрос тратит $O(\log^2 n + k)$ времени.*

Замечание. Время на ответ на запрос мы скоро улучшим.

2.5 Многомерные range-деревья

Обобщение на d измерений строится рекурсивно: главное дерево — BST по первой координате; ассоциированная структура каждой вершины — $(d-1)$-мерное range-дерево по оставшимся $d-1$ координатам. Рекурсия заканчивается при $d = 2$.

Запрос делается аналогично: в главном дереве выбираем $O(\log n)$ канонических подмножеств, в каждом из них делаем $(d-1)$-мерный запрос. На i-м уровне рекурсии имеется $O(\log^i n)$ канонических подмножеств.

Теорема. *d-мерное range-дерево для n точек использует $O(n \log^{d-1} n)$ памяти, строится за $O(n \log^{d-1} n)$, запросы отвечаются за $O(\log^d n + k)$.*

Доказательство. Обозначим $T_d(n)$ — время построения для d измерений и $Q_d(n)$ — время запроса для d измерений (без учёта возврата точек, то есть только на нахождение канонических подмножеств по всем размерностям).

Построение: $T_d(n) = O(n \log n) + O(\log n) \cdot T_{d-1}(n)$, первое слагаемое из построения BST, а второе поскольку на каждом из $O(\log n)$ уровней главного дерева точка входит ровно в одну ассоциированную структуру. С учётом $T_2(n) = O(n \log n)$ по индукции $T_d(n) = O(n \log^{d-1} n)$. Память аналогично.

Запрос: $Q_d(n) = O(\log n) + O(\log n) \cdot Q_{d-1}(n)$. Первое слагаемое из поиска по дереву по первой координате, второе — $O(\log n)$ вызовов запроса меньшей размерности. $Q_2(n) = O(\log^2 n)$ по индукции $Q_d(n) = O(\log^d n)$. Осталось прибавить $O(k)$ на возвращение значений точек. □

2.6 Fractional Cascading и Layered Range-Tree

Улучшим время на запрос для плоскости до $O(\log n + k)$, назовем эту структуру *2D layered range-tree*, и используем ее для рекурсивного построения, как в range-tree. Из анализа времени на запрос для range-tree видно, что в $\mathbb{R}^d$ время на запрос улучшится до $O(\log^{d-1} n + k)$.

Сначала обсудим **fractional-cascading**. Эта техника опирается на следующее соображение. Пусть у нас есть два отсортированных массива чисел A_1 и A_2, причем A_2 является подмножеством A_1. Мы хотим ответить на обычный 1D запрос $[x : x']$. Можно было бы для каждого из массивов вызвать бинарный поиск для x, а после обойти все числа правее и вернуть их, пока они меньше x', поэтому мы потратим $O(\log|A_1|+\log|A_2|+k_1+k_2)$ времени. Но мы хотим воспользоваться знанием $A_1 \supset A_2$! Давайте делать не так: для каждого элемента μ в A_1 будем хранить ссылку на первый элемент A_2, не меньший μ. Теперь вызовем двоичный поиск для нижней границы в A_1, начнем идти вправо от найденного числа t, пока не встретим в A первый элемент $> x'$. Аналогично можно сделать и для A_2, но в этот раз мы находим первый элемент $\geq x$ бесплатно — просто берем ссылку, соответствующую t, после чего идти направо, пока не дойдем до числа $> x'$. Таким образом, для поиска в A_1 мы потратили $O(\log|A_1| + k_1)$, а в A_2 — $O(1 + k_2)$.

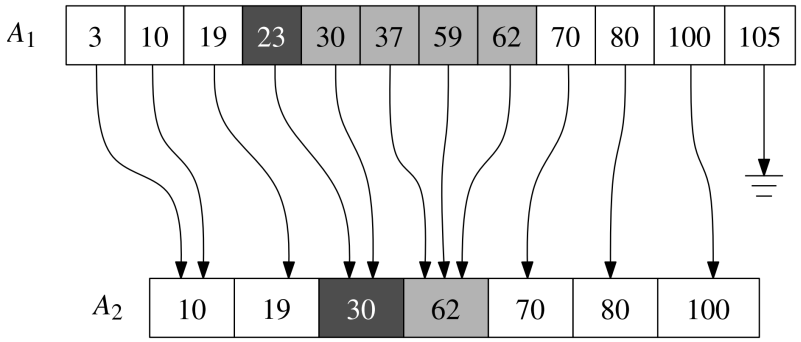

Рис. 6: Делаем запрос [20:65] для массивов A_1 и A_2

Теперь о применимости этой идеи к range-деревьям. Напомню, сейчас рассматриваем 2D реализацию. Заметим, что $P(\nu) = P(lc(\nu)) \bigsqcup P(rc(\nu))$ (lc, rc - левый и правый ребенок). Тогда давайте заменим каждый BST $T_{\text{assoc}}(\nu)$ на отсортированный массив $A(\nu)$. В каждой ячейке $A(\nu)[i]$ помимо точки храним два указателя — на ячейки $A(\text{lc}(\nu))$ и $A(\text{rc}(\nu))$, хранящие точку с наименьшей y-координатой $\geq$ текущей (прямо как в fractional-cascading). Такую модифицированную версию range-tree называют **Layered range-tree**.

Как работает запрос. Как и раньше, мы ищем те $O(\log n)$ вершин, чьи канонические подмножества представляют интересующий отрезок по x координате, но теперь мы делаем один двоичный поиск по y. В ν_{split} делаем двоичный поиск по y за $O(\log n)$. Далее, спускаясь по главному дереву как в 1DRangeQuery, поддерживаем первую позицию со значением $\geq y$ в массиве каждой посещённой вершины — она обновляется за $O(1)$ переходом по указателю. Для выбранной вершины ν сразу знаем стартовую позицию в $A(\nu)$, остаётся пройти вправо, пока $A(\nu)[i].y \leq y'$ — это $O(1 + k_\nu)$. Просуммировав по $O(\log n)$ вершин, получим время $O(\log n + k)$.

В d измерениях fractional-cascading убирает один множитель $\log n$, давая время запроса $O(\log^{d-1} n + k)$ при том же времени построения и памяти ($O(n \log^{d-1} n)$).

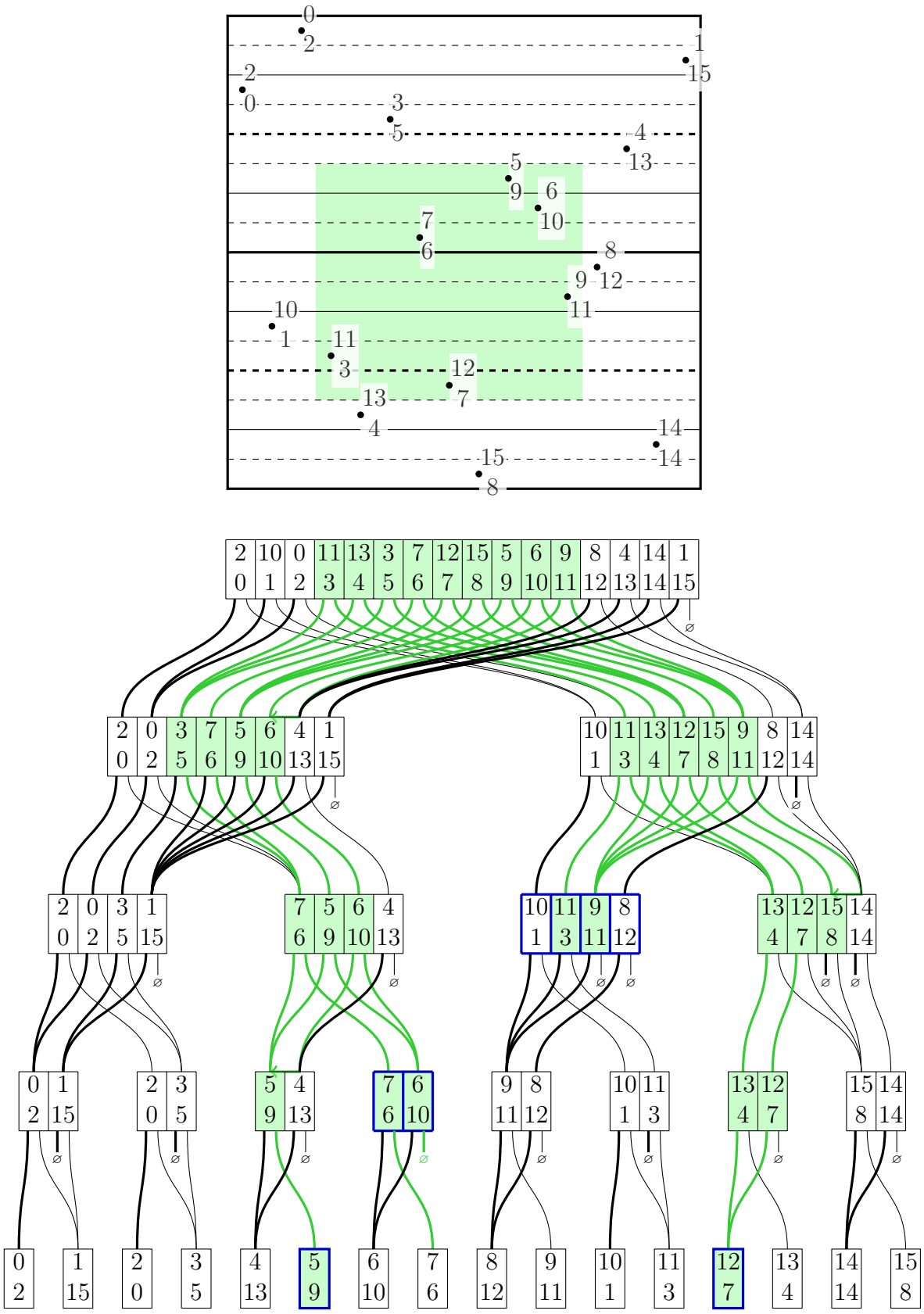

Рис. 7: Множество точек в $\mathbb{R}^2$ и запрос $[5, 12] \times [3, 9]$ (вверху, координаты «матричные»); 2D range-дерево и переходы по ссылкам, соответствующие обработке этого запроса (внизу). Синим обведены вершины, накрывающие диапазон запроса по первой координате. Зелёная заливка — точки, принадлежащие диапазону по второй координате.

2.7 Совпадающие координаты

До сих пор предполагалось, что все x и y координаты различны. Это ограничение снимается переходом в *composite-number space*.

Для построения дерева по сути только требовалось, чтобы мы могли сравнивать числа в нем. Каждую точку $p = (p_x, p_y)$ заменяем на $\hat{p} = ((p_x|p_y), (p_y|p_x))$, где $(a|b)$ — пара вещественных чисел с лексикографическим порядком: $(a|b) < (a'|b') \Leftrightarrow a < a'$ или ($a = a'$ и $b < b'$). При такой замене и первые, и вторые координаты всех точек попарно различны, что и нужно для построения дерева.

Запрос $[x : x'] \times [y : y']$ преобразуется в $[(x| - \infty) : (x'| + \infty)] \times [(y| - \infty) : (y'| + \infty)]$.